

2.3 Email

Electronic Mail in the Internet



Kurose & Ross, 8th Ed.



Email — The Internet's First Killer Application

Section 2.3: Introduction

Historical Significance

Email was the most popular application when the Internet was in its infancy.

It predates the Web, streaming video, and social media.

Despite decades of competition, email remains one of the most widely used communication systems in the world.

Why Email Is Interesting for Networking

Unlike HTTP (one protocol, one job), Internet email uses several cooperating application-layer protocols — each responsible for a different piece of the pipeline.

Understanding email = understanding how multiple protocols work together.

Email vs. Postal Mail:

Asynchronous · Instantaneous (seconds, not days) · Essentially free · Distributable to thousands at once · Rich: attachments, HTML, images, hyperlinks

Email is more complex than the Web — it uses at least 3 protocols cooperating to deliver a single message

Three Major Components of Internet Email

Section 2.3: Overview

① User Agents

Software that allows users to: read, compose, reply, forward, and send messages.

Examples:

- Microsoft Outlook
- Apple Mail
- Gmail (browser)
- Gmail App (smartphone)
- Thunderbird

② Mail Servers

Core infrastructure of email.

Each user has a mailbox on a mail server.

Responsibilities:

- Manage user mailboxes
- Run both client and server sides of SMTP
- Hold messages in queue if destination is unreachable
- Retry delivery every ~30 minutes for several days

③ SMTP

Simple Mail Transfer Protocol.

The principal application-layer protocol for Internet email.

Transfers messages from sender's mail server to recipient's mail server.

Runs over TCP · Port 25
Push protocol
Defined in RFC 5321

Mail servers run BOTH sides of SMTP — acting as SMTP client when sending, SMTP server when receiving

The Journey of an Email: Alice to Bob

Section 2.3: Overview

1

Alice composes & sends

Alice uses her user agent (e.g., Outlook). She types bob@hamburger.edu, writes the message, and clicks Send.

2

User agent → Alice's server

Alice's user agent delivers the message to Alice's mail server (crepes.fr) and places it in the outgoing message queue.

SMTP or HTTP

3

Alice's server → Bob's server

The SMTP client on Alice's server opens a direct TCP connection on port 25 to Bob's mail server (hamburger.edu) and pushes the message.

SMTP (TCP port 25)

4

Bob's mailbox filled

Bob's mail server receives the message and deposits it in Bob's mailbox. If Bob's server was down, Alice's server retries every 30 min.

5

Bob retrieves his mail

At his convenience, Bob uses his user agent to retrieve messages from his mailbox via HTTP or IMAP.

HTTP or IMAP

Why the Two-Step Delivery Process?

Section 2.3: Overview

Alice's user agent sends to Alice's server first — not directly to Bob's server

Direct Delivery (no relay) — What Could Go Wrong

If Alice's user agent tries to deliver directly to Bob's mail server:

- Bob's server might be temporarily down.
- Alice's user agent would fail immediately — no recourse.
- The message would be lost.

User agents are ephemeral — they are only online when the user opens the app.

Relay Through Alice's Server — Why It Works

Alice's mail server is always on.

If Bob's server is down:

- Alice's server holds the message in a queue.
- It retries delivery every ~30 minutes.
- After several days of failure: removes the message and notifies Alice with a bounce email.

Responsibility for delivery shifts to the always-on infrastructure.

No intermediate relay servers — even Hong Kong → St. Louis is a direct TCP connection between the two mail servers

SMTP — Overview and History

Section 2.3.1: SMTP

SMTP = Simple Mail Transfer Protocol · RFC 5321 · TCP Port 25 · Push

Key Properties

- Defined in RFC 5321
- Original RFC dates back to 1982 — among the oldest Internet protocols in active use
- Runs over TCP (port 25)
- Push protocol: sender pushes message to recipient's server without being asked
- Uses persistent connections
- Plain ASCII commands — human readable

The 7-bit ASCII Limitation

SMTP restricts ALL message content (body AND headers) to 7-bit ASCII.

This made sense in 1982 when bandwidth was scarce and no one sent large attachments.

Today this is a legacy constraint — binary data (images, audio, PDFs) must be encoded to ASCII before sending and decoded after receipt.

Solution: MIME (Multipurpose Internet Mail Extensions)

Contrast with HTTP: HTTP has no ASCII restriction — it transfers raw binary directly. SMTP pre-dates HTTP by a decade.

SMTP Dialogue: A Complete Transcript

Section 2.3.1: SMTP

Client: crepes.fr → Server: hamburger.edu (TCP connection established)

S:	220 hamburger.edu	← Server ready
C:	HELO crepes.fr	← Client introduces itself
S:	250 Hello crepes.fr, pleased to meet you	
C:	MAIL FROM: <alice@crepes.fr>	← Declare sender
S:	250 alice@crepes.fr ... Sender ok	
C:	RCPT TO: <bob@hamburger.edu>	← Declare recipient
S:	250 bob@hamburger.edu ... Recipient ok	
C:	DATA	← Start message body
S:	354 Enter mail, end with "." on a line by itself	
C:	Do you like ketchup? How about pickles?	← Message body
C:	.	← End of message (lone period)
S:	250 Message accepted for delivery	
C:	QUIT	← Close connection
S:	221 hamburger.edu closing connection	

SMTP Commands and Reply Codes

Section 2.3.1: SMTP

Client Commands

HELO	Introduce client to server (start handshake)
MAIL FROM	Declare the sender's email address
RCPT TO	Declare the recipient's email address
DATA	Signal start of message body
QUIT	Close the TCP connection
CRLF .CRLF	A lone period on its own line — marks end of message body

Server Reply Codes

220	Service ready — server announces itself
221	Service closing transmission channel
250	Requested action completed successfully
354	Start mail input — send message body now
421	Service not available (try later)
550	Mailbox unavailable — recipient not found

Tip: Use `telnet serverName 25` to interact directly with any SMTP server and type these commands by hand.

SMTP vs. HTTP — Two Client Models

Section 2.3.1: SMTP

SMTP pushes — HTTP pulls. Which protocol your client uses depends entirely on what kind of client it is.

Dedicated Mail Clients — SMTP

Outlook · Thunderbird · Apple Mail

- Client speaks SMTP directly (port 587)
- AUTH uses your personal credentials
- SMTP logic runs on your machine
- Persistent connections: multiple messages share one TCP session (MAIL FROM → . → QUIT)

Your machine opens two TCP connections:
TCP:443 (HTTPS auth) + TCP:587 (SMTP)

Web / App Clients — HTTP

Gmail (browser) · Outlook.com · Yahoo Mail

- Browser speaks only HTTP/HTTPS (port 443)
- HTTP POST carries message to web server
- Web server translates to SMTP internally
- AUTH uses Google's server credentials
- SMTP logic runs on Google's servers

Your machine opens only one TCP connection:
TCP:443 (HTTPS) ← nothing else

The client model determines the protocol stack — dedicated clients own their SMTP session; web clients delegate it entirely

Mail Message Format (RFC 5322)

Section 2.3.2: Mail Message Formats

Header lines + blank line (CRLF) + message body (ASCII)

Example message:

From: `alice@crepes.fr` *Required*

To: `bob@hamburger.edu` *Required*

Subject: `Searching for the meaning of life.` *Optional*

Date: `Mon, 3 Oct 2024 09:23:00` *Optional*

(blank line) *← blank line (CRLF)*

Do you like ketchup? *Body*

How about pickles?

Header fields:

From:	<i>Required</i>	Sender's email address
To:	<i>Required</i>	Recipient's email address
Subject:	<i>Optional</i>	Brief description of the message
Date:	<i>Optional</i>	When the message was composed
Reply-To:	<i>Optional</i>	Address to use when replying (can differ from From:)
CC:	<i>Optional</i>	Carbon copy recipients
BCC:	<i>Optional</i>	Blind carbon copy recipients

IMPORTANT: header fields (From: / To:) ≠ SMTP commands (MAIL FROM: / RCPT TO:) — they look similar but serve different purposes

SMTP Commands vs. Message Header Lines

Section 2.3.2: Mail Message Formats

These look similar — but they are fundamentally different things

SMTP Commands (in the SMTP dialogue)

MAIL FROM: <alice@crepes.fr>
RCPT TO: <bob@hamburger.edu>

These are PART OF THE SMTP PROTOCOL.

They are handshaking commands that tell the server how to route the message.

They are typed during the SMTP dialogue BEFORE the message body is sent.

They are NOT included in the message the recipient sees.

Message Header Lines (inside the message)

From: alice@crepes.fr
To: bob@hamburger.edu
Subject: ...

These are PART OF THE MESSAGE ITSELF.

They are metadata displayed to the recipient by their email client.

They appear inside the DATA section — after the SMTP DATA command.

They DO NOT have to match the SMTP commands — spammers exploit this to forge the apparent sender.

Analogy: The SMTP commands are like the label on the postal envelope (routing info). The message headers are like what is written on the letter inside.

MIME — Multipurpose Internet Mail Extensions

Section 2.3.2: Mail Message Formats

Problem: SMTP allows only 7-bit ASCII — how do we send images, PDFs, audio, or binary files?

MIME Solution

MIME adds extra header lines to declare the content type and encoding:

Content-Type: image/jpeg
Content-Transfer-Encoding: base64

Binary data is encoded with base64:

- Every 3 bytes → 4 ASCII characters
- Size increases ~33%
- But the data is now valid 7-bit ASCII

At the receiver, the client decodes base64 back to binary and presents the attachment.

Common MIME Content Types:

text/plain	Plain text body
text/html	HTML-formatted email
image/jpeg	JPEG image attachment
audio/mpeg	MP3 audio file
application/pdf	PDF document
multipart/mixed	Message with multiple parts (text + attachments)

MIME is transparent to the user — email clients handle encoding/decoding automatically

Mail Access: Why Can't Bob Use SMTP?

Section 2.3.3: Mail Access Protocols

SMTP is a PUSH protocol — retrieving messages is a PULL operation

The Problem

Bob's messages are stored on a mail server.
Bob runs a user agent on his laptop or phone.

SMTP only pushes messages from sender to receiver.
SMTP has no command for:
"Please give me my messages"

SMTP cannot retrieve mail.
A different protocol is needed for the last leg.

Why Not Run a Server on Bob's PC?

If Bob's mail server ran on his own laptop:

- His laptop must be always on and always connected — new messages can arrive at any time.
- This is completely impractical for most users.

Solution: separate the always-on mail server from Bob's user device.

Bob's user agent retrieves messages from the server when he is ready to read them.

Two solutions for retrieval: HTTP (web-based email / smartphone apps) and IMAP (dedicated mail clients)

Mail Retrieval via HTTP

Section 2.3.3: Mail Access Protocols

If Bob uses Gmail, Outlook.com, Yahoo Mail, or a smartphone app — his client uses HTTP

How It Works

Bob's browser or app connects to the mail server over HTTP (or HTTPS).

The mail server must expose two interfaces:

- An SMTP interface — to receive messages from other mail servers
- An HTTP interface — to serve messages to Bob's browser / app

Bob reads, composes, and manages his email entirely through web requests.

Examples & When It Is Used

- Gmail (browser or Android/iOS app)
- Outlook.com
- Yahoo Mail
- Corporate web portals

This is the most common approach today.
The user needs no special software beyond a browser or the app.

HTTPS (port 443) is used in practice for security.

Full protocol stack for web-based email:



IMAP — Internet Mail Access Protocol

Section 2.3.3: Mail Access Protocols

IMAP (RFC 3501) — used by dedicated mail clients: Outlook, Thunderbird, Apple Mail

IMAP Features

Messages remain stored on the server — not just downloaded.

Bob can:

- Organize messages into server-side folders
- Move messages between folders
- Mark messages as read, important, or flagged
- Search through messages on the server
- Delete messages

All changes are synchronized across all of Bob's devices.

The Key Advantage: Multi-Device Synchronization

- Bob reads an email on his phone → laptop also shows it as read
- Bob creates folder 'Projects' on laptop → appears on phone too
- Bob deletes a message on tablet → gone from all devices
- Bob flags an important email on work PC → visible on all devices

This server-side synchronization is the critical advantage of IMAP in the era of multiple devices.

POP3 vs. IMAP: Legacy vs. Modern

Section 2.3.3: Mail Access Protocols

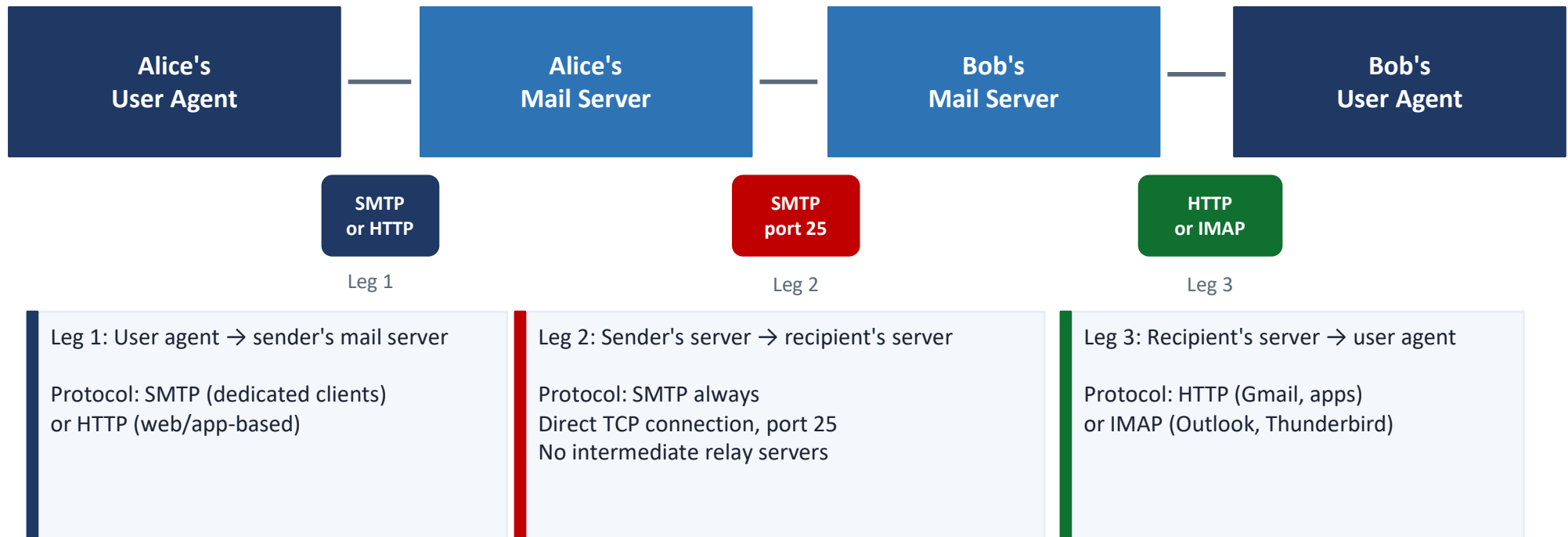
Before IMAP became dominant, the standard retrieval protocol was POP3 (RFC 1939)

Dimension	POP3 (legacy)	IMAP (modern)
RFC	RFC 1939	RFC 3501
Message storage	Downloads to device, deletes from server	Stored on server (always)
Folders	Local only — no server-side folders	Server-side folders, synchronized
Multi-device sync	None — device A and device B see different states	Full sync across all devices
Offline access	Yes — messages downloaded locally	Yes — messages cached locally
Bandwidth use	Downloads all messages	Downloads only what is viewed
Designed for	Single device, limited disk era (1980s)	Multiple devices, modern usage
Still used?	Legacy corporate environments	Standard for dedicated clients

The Complete Email Protocol Architecture

Section 2.3.3: Mail Access Protocols

Three legs — three protocols — one message from Alice to Bob



SMTP & IMAP — Separate Architectural Roles

Section 2.3.3: Mail Access Protocols

SMTP = PUSH (sending) · IMAP = PULL (reading) — each protocol has exactly one responsibility

SMTP — Client → Server

[Outlook]
SMTP | (587/465) → SEND
[Mail Server]
SMTP | (port 25) → FORWARD

IMAP — Server → Client

[Destination Mail Server]
IMAP | (port 993) → READ
[Bob's Outlook]
fetch / delete / move / mark as read

Simple rule: SMTP always pushes messages forward — IMAP only manages what is already stored on a server

Why Can't IMAP Be Used for Sending?

Section 2.3.3: Mail Access Protocols

IMAP was designed exclusively for access and management of messages already stored on a server

IMAP Operations — Management Only

All IMAP commands operate on messages that already exist on the server:

- FETCH — retrieve a message or header
- STORE — change flags (read, flagged, deleted)
- COPY — copy message to another folder
- MOVE — move message between folders
- SEARCH — search mailbox by criteria
- DELETE — mark message for deletion

None of these commands can inject a new message into the delivery chain.

SMTP — Delivery Chain Responsibility

SMTP was built specifically to move messages between points:

- MAIL FROM — declare envelope sender
- RCPT TO — declare envelope recipient
- DATA — inject the full message body

This is what makes SMTP a transfer protocol — it has the commands to introduce a brand new message to the delivery infrastructure.

IMAP has no equivalent of DATA for outbound injection.

The protocols are complementary, not interchangeable — IMAP manages the mailbox; SMTP drives the delivery pipeline

SMTP Ports — Client Submission vs. Server-to-Server

Section 2.3.1: SMTP

Not all SMTP traffic is equal — client-to-server and server-to-server use different ports intentionally

Port 587 — SUBMISSION

Name = Who uses it
Value = End-user clients (Outlook, Thunderbird, mobile apps)

Client → Mail Server: SMTP AUTH required
Introduced by RFC 4409 to separate user traffic

Port 465 — SMTPS

Name = Who uses it
Value = Clients requiring implicit TLS from connection start

Client → Mail Server: TLS wraps the entire session
Older standard, still widely supported by providers

Port 25 — Classic SMTP

Name = Who uses it
Value = Mail servers communicating with other mail servers

Server → Server: no end-user auth expected
Many ISPs block outbound port 25 to prevent spam

Port 587 (Submission) was introduced specifically to separate client traffic from inter-server traffic — it allows mail servers to enforce authentication and per-user policies (rate limiting, spam filters) exclusively on messages coming from human users.

Browser Clients Never Speak SMTP

Section 2.3.3: Mail Access Protocols

The browser is a generic HTTP client — it has no SMTP implementation and needs none

When you click Send in Gmail (browser)

```
[ Browser ]  
| HTTP POST /mail/send  
▼  
[ Web Server Google / nginx ]  
| reverse proxy  
▼  
[ Gmail Backend (Python/Java/Go) ]  
| smtpplib.sendmail(...) ← runs on Google  
▼  
[ Gmail Mail Server (SMTP) ]  
| SMTP (port 25 or internal)  
▼  
[ Destination Mail Server ]
```

When you click Send in Outlook

```
[ Outlook ]  
| SMTP (port 587)  
| AUTH with YOUR credentials  
▼  
[ Your Mail Server ]  
| SMTP (port 25)  
▼  
[ Destination Mail Server ]
```

Sending via Gmail in a browser = submitting an HTTPS form — your machine never opens an SMTP connection

Gmail in Browser — Step-by-Step Flow

Section 2.3.3: Mail Access Protocols

1**You press Send in Gmail**

Browser generates an HTTP POST request with JSON body: { to, subject, body }. No SMTP involved — this is a standard web API call on port 443.

2**nginx (Google Web Server) receives the HTTPS request**

Terminates TLS, authenticates your session cookie, and routes the request to the Gmail backend application via reverse proxy.

3**Gmail backend application constructs the SMTP session**

Backend code (Python smtplib, Java JavaMail, etc.) opens an SMTP connection to the internal Gmail mail server. AUTH uses Google's server credentials — not yours.

4**Gmail Mail Server delivers via SMTP**

Standard SMTP session: MAIL FROM (your address) → RCPT TO → DATA → lone period. Direct TCP connection on port 25 to the recipient's mail server.

5**Recipient's mail server accepts the message**

Identical to any other SMTP delivery. The recipient has no way to know whether you used Outlook or a browser — the SMTP envelope looks the same.

Key insight: AUTH SMTP uses Google's server credentials — your email password is never sent to the mail server directly

Outlook vs. Browser — Protocol Comparison

COMPARISON

Section 2.3.3: Mail Access Protocols

	Outlook (dedicated client)	Browser (Gmail)
Protocol: client → server	SMTP (port 587)	HTTP/HTTPS (port 443)
Who speaks SMTP	Outlook directly	Gmail backend (on Google servers)
SMTP AUTH credentials	Your personal credentials	Google server credentials
Where SMTP logic runs	On your machine	On Google's servers
Your machine's TCP footprint	TCP:443 + TCP:587	TCP:443 only
Retrieval protocol	IMAP (port 993)	HTTP/HTTPS (port 443)
Offline capability	Yes — full local cache	Limited (Service Worker)

From your machine's perspective, using Gmail in a browser is indistinguishable from visiting any other website

Network Footprint — Outlook vs. Browser

Section 2.3.3: Mail Access Protocols

When using Gmail in a browser, your machine makes exclusively HTTPS connections — no SMTP, ever

Outlook — TCP connections from your machine

Sending:

TCP:443 → HTTPS authentication / OAuth token

TCP:587 → SMTP SUBMISSION (your message)

Receiving:

TCP:993 → IMAP over TLS

If you capture your network traffic with Wireshark, you will see dedicated SMTP and IMAP sessions distinct from your browser traffic.

Browser (Gmail) — TCP connections from your machine

Sending:

TCP:443 → HTTPS only (HTTP POST with message)

Receiving:

TCP:443 → HTTPS only (HTTP GET polling / WebSocket)

A packet capture of your browser shows zero SMTP or IMAP traffic. All mail operations are HTTP calls. The SMTP session happens entirely within Google's internal network — invisible to you.

The browser is a universal HTTP client — the SMTP complexity is fully abstracted away by the web application layer

Summary

2.3 Electronic Mail in the Internet

- Internet email has 3 components: user agents (clients), mail servers, and SMTP.
- Message journey: user agent → Alice's server → (SMTP) → Bob's server → user agent. Two-step for reliability.
- SMTP (RFC 5321): push protocol, TCP port 25, 7-bit ASCII restriction, persistent connections.
- SMTP dialogue: HELO · MAIL FROM · RCPT TO · DATA · lone period · QUIT. Numeric reply codes (220, 250, 354, 221).
- Message format (RFC 5322): From/To/Subject header lines + blank line + body. Header ≠ SMTP commands.
- MIME extends SMTP for binary content: base64 encoding converts binary → ASCII. Adds Content-Type and Content-Transfer-Encoding headers.
- SMTP is a PUSH protocol — retrieval is a PULL operation, so a separate protocol is needed.
- HTTP retrieval: used by web-based email (Gmail) and smartphone apps. Mail server needs both SMTP and HTTP interfaces.
- IMAP (RFC 3501): server-side storage, folders, and full multi-device synchronization. Dominant for dedicated clients.
- POP3 is the legacy alternative: downloads and deletes. No sync. Impractical for multi-device use.